CLINICQUEUE

MVP BUILD SPECIFICATION

Architect → Builder handoff for Google Antigravity

Mission: Build a polished, demo-ready hospital queue management prototype that behaves like a small real system—not a collection of static screens.

Role split

• **Architect (ChatGPT):** defines product boundaries, requirements, architecture, data model, workflow, acceptance tests, and build sequence.

• **Builder (Antigravity):** inspects the workspace, implements the application, runs it, tests it, fixes defects, and documents how to launch it.

Target stack: Java 21 · JavaFX · Maven · SQLite · JDBC · JUnit 5

Target: classroom presentation MVP · single-computer demo · fictional data only

Read this document as the implementation contract. If a requirement is ambiguous, choose the smallest reliable implementation that preserves the stated scope; document the choice instead of expanding scope.

1. Product definition

ClinicQueue helps a receptionist register a patient, assign a doctor and department, issue a daily token, and maintain a live doctor-specific queue. A doctor/operator can call the next patient and update the consultation status. The dashboard makes the current queue understandable at a glance.

Presentation story

- A patient arrives and is registered in under a minute.
- The system assigns a unique token for the selected doctor and date.
- The token appears in that doctor's queue and the dashboard counts update.
- The operator calls the next waiting patient, then marks them IN_CONSULTATION and COMPLETED.
- The estimated wait for waiting patients updates using a transparent, approximate rule.
- Close and relaunch the application: saved records remain available.

Successful MVP: a judge can follow this end-to-end story in a live run without manually editing the database or relying on internet access.

2. Hard scope boundaries

Build in MVP	Do not build in MVP
Patient registration with validation	Diagnosis, prescriptions, clinical decision support
Doctor/department selection from seeded setup data	Real hospital integrations or production deployment
Appointment/walk-in registration as a queue visit	SMS/email/WhatsApp notifications
Daily doctor-specific token numbering	Cloud backend, multi-device sync, online accounts
Queue states and Call Next	Billing, pharmacy, insurance, medical records
Dashboard, approximate wait, search/filter	AI predictions or claims of accurate wait times
SQLite persistence, tests, demo data/reset path	Use of real patient information

3. Technology and execution constraints

Area	Requirement
Language	Java 21; use standard Java features, avoid unnecessary frameworks.
UI	JavaFX desktop interface. FXML optional; a code-built UI is acceptable.
Build	Maven project with documented JavaFX run command and required plugins.
Database	SQLite local database file; JDBC for all persistence.
Tests	JUnit 5 for queue logic, token allocation, status transitions, and validation.
Runtime	Offline-capable after dependencies are installed; no external API required.
Design	Clean, consistent, resizable desktop UI; no dead buttons.
Data	Fictional sample patients only. Provide clearly labeled demo seed/reset.

4. Roles and user journeys

MVP roles

- **Receptionist/operator:** registers visits, assigns a doctor, views queues, searches patients, cancels an eligible visit.
- **Doctor/operator view:** views a doctor-specific queue, calls the next patient, starts consultation, completes it.
- **Admin:** not a login role in MVP; doctors and departments are pre-seeded.

Primary journey

```
Launch app
  → Dashboard (today's totals and per-doctor queue summary)
  → Register Visit
      → patient name + optional phone + department + doctor + visit type
      → validate inputs
      → save patient/visit transactionally
      → allocate next daily token for that doctor
  → Queue Board
      → select doctor; view status-filtered queue
      → Call Next: earliest eligible WAITING visit
      → Start Consultation / Complete
  → Dashboard refreshes counts and approximate wait
```

5. Functional requirements (acceptance contract)

ID	Requirement / acceptance criterion
FR-01	Register patient with required name and optional phone. Trim whitespace and reject blank names.
FR-02	Create a visit for selected doctor/date and type APPOINTMENT or WALK_IN. Both types enter the same queue in MVP; label this clearly.
FR-03	Generate sequential token per doctor per calendar date (e.g. D01-2026-10-20-001). Never duplicate after restart.
FR-04	Show token, patient name, visit type, creation time, status. Display only necessary personal data.
FR-05	Statuses: WAITING, IN_CONSULTATION, COMPLETED, CANCELLED.
FR-06	Call Next selects oldest WAITING visit for selected doctor/date and marks IN_CONSULTATION. If none, show a helpful message.
FR-07	Complete only from IN_CONSULTATION. Cancel only from WAITING. Block invalid transitions with explanation.
FR-08	Prevent a second IN_CONSULTATION visit for the same doctor at a time.
FR-09	Dashboard shows counts for all four statuses for selected date and per-doctor summaries.
FR-10	Search by patient name/token; filter queue by status and doctor.
FR-11	Estimate wait as eligible WAITING visits ahead × average duration (default 10 min). Label APPROXIMATE and show assumption.
FR-12	Persist patients, visits, statuses, doctors, departments in SQLite across restarts.
FR-13	Provide sample data and a confirmed reset/reseed action. Warn that reset deletes local demo records.
FR-14	Show success/error feedback. No silent failures, user-facing stack traces, or nonfunctional buttons.
FR-15	Use prepared statements, foreign keys, DB constraints, and transactions for visit/token creation.

6. Non-functional requirements

- **Reliability:** core workflow runs from clean checkout; failed database writes must not create partial visits.
- **Usability:** obvious primary actions, readable status labels, clear validation, confirmation for destructive actions.
- **Maintainability:** separate UI, business logic, and database code; no SQL/queue rules in JavaFX handlers.
- **Performance:** immediate-feeling interactions with a few hundred local demo visits; avoid blocking JavaFX thread.
- **Privacy:** fictional data only; no secrets committed; prototype is not production healthcare software.
- **Testability:** business rules testable without launching JavaFX where practical.

7. Architecture (required)

```
JavaFX Views
  ■■■ DashboardView
  ■■■ RegisterVisitView
  ■■■ QueueBoardView
          ▼
Controllers (UI coordination only)
          ▼
Services (business rules)
  ■■■ VisitService      → registration + transaction boundary
  ■■■ QueueService      → Call Next + status transitions
  ■■■ TokenService      → daily token allocation
  ■■■ DashboardService  → counts + wait estimates
          ▼
```

```
Repositories (JDBC / prepared statements)
  ■■■ PatientRepository
  ■■■ VisitRepository
  ■■■ DoctorRepository
  ■■■ DepartmentRepository
        ▼
SQLite database (clinicqueue.db)
```

Architecture rules

- Controllers must not execute SQL directly.
- Repositories must not contain UI code.
- Services validate rules before writing.
- Token allocation and visit insertion happen in one transaction.
- Use a shared DB configuration utility; enable SQLite foreign keys on every connection.
- Schema initialization must be safe to run on every startup.

8. Data model and database constraints

Table	Important columns / constraints
departments	id PK; name UNIQUE; active flag
doctors	id PK; department_id FK; display_name; active flag
patients	id PK; full_name NOT NULL; phone nullable; created_at
visits	id PK; patient_id FK; doctor_id FK; visit_date; visit_type; token_number; status; created_at; updated_at
settings (optional)	key PK; value, e.g. average_consultation_minutes

Integrity rules

- UNIQUE(doctor_id, visit_date, token_number).
- Constrain status to WAITING, IN_CONSULTATION, COMPLETED, CANCELLED.
- Constrain visit_type to APPOINTMENT or WALK_IN.
- Enable foreign keys and use NOT NULL where appropriate.
- Store timestamps consistently and document local-date/time behavior.
- Allocate token by reading the maximum token for doctor/date inside a transaction; handle constraint errors safely.

9. Queue rules

Action	Rule
Register	Create visit as WAITING and allocate next doctor/date token in one transaction.
Call Next	If doctor has IN_CONSULTATION visit, refuse. Otherwise choose oldest WAITING by created_at then id.
Complete	Only IN_CONSULTATION → COMPLETED.
Cancel	Only WAITING → CANCELLED; cancelled visits do not count as waiting ahead.
Wait estimate	Waiting position × average duration. Optionally add current consultation remaining time only if implemented transparently; otherwise omit.
Order	created_at then id is primary ordering; token is display identifier.

Do not claim wait time is predictive or guaranteed. It is a simple illustrative estimate for a classroom prototype.

10. UI specification

Screen	Required content and interactions
Dashboard	App title; date defaulting to today; four status counts; per-doctor summaries; Register Visit and Queue Board navigation.
Register Visit	Required patient name; optional phone; department; doctor filtered by department; visit type; Register; validation; success state with token.
Queue Board	Doctor dropdown; status filter; search; queue table; status labels; Call Next; Complete/Cancel actions enabled only when valid.
Dialogs/states	Confirm reset/reseed and cancellation; helpful empty states; errors with next-step guidance.

Visual direction

- Professional clinic-operations style: navy/blue base, neutral backgrounds, clear status chips, consistent typography.
- Make token and queue status prominent. Never rely on color alone; show text labels.
- Prefer one main window with simple navigation; avoid unnecessary animations/chart libraries.
- Design for a 1366×768 laptop presentation and ensure resize does not clip controls.
- Every visible interactive control must work; remove placeholder buttons.

11. Suggested project structure

```
ClinicQueue/
■■■ pom.xml
■■■ README.md
■■■ docs/
■   ■■■ ARCHITECTURE.md
■   ■■■ DEMO_SCRIPT.md
■   ■■■ TEST_PLAN.md
■■■ src/main/java/in/clinicqueue/
■   ■■■ ClinicQueueApp.java
■   ■■■ config/
■   ■■■ model/
■   ■■■ controller/
■   ■■■ service/
■   ■■■ repository/
■   ■■■ database/
■   ■■■ util/
■■■ src/main/resources/in/clinicqueue/
■   ■■■ view/             # optional FXML
■   ■■■ css/
■■■ src/test/java/in/clinicqueue/
    ■■■ service/
    ■■■ repository/
```

Package names may vary, but keep separation of concerns. Do not create a large framework for a small MVP.

12. Build plan and implementation order

Phase	Builder deliverable	Done when
0 Inspect	Inspect workspace, Java/Maven versions and existing files. Plan before overwriting anything.	No user files overwritten without review.
1 Skeleton	Maven JavaFX app, entry point, theme, navigation, README run instructions.	App launches from documented command.
2 Persistence	SQLite schema, seed doctors/departments, repositories, foreign keys.	Clean DB initializes; seed is idempotent.
3 Registration	Form, validation, visit service, transactional token allocation.	Distinct tokens; restart preserves records.
4 Queue engine	Queue board, Call Next, Complete, Cancel, valid transitions.	Rules enforced and visible.
5 Dashboard	Counts, doctor summaries, search/filter, approximate wait.	Counts match persisted rows.
6 Quality	JUnit tests, error handling, UI polish, confirmed reset/reseed.	Tests pass; no dead buttons.
7 Handoff	README, architecture notes, test plan, demo script, limitations.	Teammate can launch and present.

13. Acceptance test checklist

- ■ Fresh start initializes schema and seeded departments/doctors.
- ■ Blank and whitespace-only names are rejected.
- ■ Valid visit saves and success screen shows token.
- ■ Same doctor/date visits receive different sequential tokens.
- ■ Different doctors can each start at token 001.
- ■ Records and statuses persist after restart.
- ■ Call Next selects oldest WAITING, excluding cancelled/completed visits.
- ■ Second Call Next is blocked while doctor has an IN_CONSULTATION visit.
- ■ Completing WAITING is blocked; completing IN_CONSULTATION succeeds.
- ■ Cancelling WAITING succeeds; cancelling completed visit is blocked.
- ■ Dashboard counts match queue rows and update after actions.
- ■ Search by name/token and status filters return expected results.
- ■ Wait estimate excludes cancelled/completed visits and says approximate.
- ■ Reset/reseed requires confirmation and restores known demo state.
- ■ No SQL exception/stack trace is shown to users; errors are logged.
- ■ Automated tests pass with the documented Maven command.

14. Demo data and 3-minute presentation flow

Seed a small fictional dataset on first launch or provide a one-click demo seed action. Example: 2 departments, 3 doctors, 6–8 fictional visits across WAITING, IN_CONSULTATION and COMPLETED. Clearly mark records as sample data.

Time	Presenter action	What it proves
0:00–0:25	Open dashboard; explain counts and doctor-wise view.	Operational overview.
0:25–1:00	Register “Aarav Demo” for a doctor; show token.	Validation, persistence, token generation.
1:00–1:40	Open queue; Call Next; show status change.	Queue selection and state machine.
1:40–2:10	Complete consultation; point out count update.	Cross-screen consistency.
2:10–2:35	Search by token/name; show approximate wait.	Usability and calculation.
2:35–3:00	Restart or show persistence; explain future scope.	Database persistence and scope.

15. Instructions for Google Antigravity

Antigravity should not blindly generate every file in one pass. Implement in phases, run the application and tests, inspect failures, and fix them.

- Inspect the existing workspace first. Do not delete/overwrite user files without explaining why and asking if destructive.
- Before coding, provide a concise implementation plan, environment assumptions, and blockers; then proceed using this spec.
- Implement only the MVP. Do not add cloud services, authentication, AI, SMS, or unrelated modules.
- Keep JavaFX handlers thin; put business rules in services and database access in repositories.
- Use prepared statements and transactions. Never concatenate user input into SQL.
- After each phase, run Maven compile/tests; run the desktop app where possible and fix failures before continuing.
- Do not claim a command/test passed unless it was actually executed and the result inspected.

- If GUI runtime is unavailable, run compile/tests and report that visual testing was not possible.
- Do not leave TODOs, placeholder controls, fake buttons, or hardcoded dashboard counts in the demo path.
- Use fictional data only. Do not describe the prototype as certified or suitable for actual hospital operations.
- At completion, report files changed, architecture, commands/results, demo steps, limitations, and unresolved defects.

16. Final handoff prompt (copy into Antigravity)

```
You are the implementation builder for ClinicQueue. Treat the attached “ClinicQueue MVP — Antigravity Build Specification” as the s

Goal: deliver a polished, reliable, presentation-ready Java desktop MVP using Java 21, JavaFX, Maven, SQLite, JDBC and JUnit 5.

Work method:
1. Inspect the workspace and environment before changing files. Preserve existing work.
2. Summarize the implementation plan and any blockers.
3. Build in phases: app skeleton → database/schema/seed → registration/token allocation → queue state machine → dashboard/search/
4. Keep UI, services and repositories separated. Use prepared statements, foreign keys, transactions and validation.
5. Follow all functional requirements, queue transition rules, data constraints, scope boundaries and acceptance tests in this spec
6. Run Maven compile/tests after meaningful phases and fix failures. Launch the app if the environment supports it. Never claim a t
7. Do not add out-of-scope features or fake functionality. Use fictional demo records only.
8. Finish with a working README, architecture notes, test plan and 3-minute demo script.

Definition of done: a fresh launch initializes the database; a user can register a visit, receive a unique doctor/date token, view

If a detail is unspecified, choose the smallest maintainable solution consistent with this specification and document it. Do not we
```

17. Risks and known limitations

- **Not production healthcare software:** no authentication, access-control audit, compliance review, encryption/backup strategy, or hospital integration.
- **Single-computer prototype:** SQLite is appropriate for this classroom MVP; multi-device concurrent use is future scope.
- **Wait time:** based on a fixed average duration and queue position; it can be wrong in real life.
- **Appointment semantics:** appointments enter the same queue as walk-ins; time-slot scheduling is future scope.
- **Demo safety:** use fictional names and contact details.

18. Completion report template

```
Implementation status: COMPLETE / PARTIAL
Java version:
Maven command(s) run:
Compile result:
Test result (passed / failed):
Application launch result:
Main files/modules created:
Demo seed/reset instructions:
3-minute demo steps:
Known limitations:
Unresolved defects:
Next recommended improvements:
```

Architectural priority: ship a complete, tested core workflow first. A small system that reliably demonstrates registration → token → queue → completion → dashboard → persistence is stronger than a large system with unfinished features.